

Instagram Analytics & Influencer Prediction

A complete AI-ready study guide, presentation guide, and technical reference for the notebook project.

Project purpose

Turn Instagram-style activity data into meaningful user engagement features, then use machine learning to classify engagement-based influencer status and estimate engagement per post.

What this guide covers

Project story, data tables, end-to-end workflow, feature engineering, models, evaluation, limitations, interview answers, and self-introduction script.

Project at a glance

Area	Implemented in this project
Data source	Seven CSV files representing users, posts, likes, comments, follows, photo tags, and tags.
Classification task	Predict “Influencer” vs “Not Influencer” from a user’s total likes and comments.
Regression tasks	Predict likes per post and comments per post from follower and post counts.
Models	RandomForestClassifier and two RandomForestRegressor models.
Analytics	Confusion matrix plus followers-vs-likes and posts-vs-engagement scatter plots.

Best one-line summary

I built an Instagram analytics project that combines relational social-media data, engineers engagement features, and applies random forest models for engagement-based classification and per-post engagement forecasting.

1. Project Story and Business Problem

Social-media platforms generate many connected events: users publish photos, other users like or comment on them, and users follow one another. Looking at a single table does not show who is engaging, whose content receives engagement, or what level of response a new account may get. This project joins those events into user-level features and turns them into simple predictive tasks.

The project answers two practical questions:

- **Classification:** Based on a user’s activity (likes and comments made), does the project place that user above the median engagement threshold?
- **Forecasting:** Given an account’s follower count and number of posts, what likes per post and comments per post can be expected?

End-to-end workflow

users, photos, likes, comments, follows	Identify keys: user_id, photo_id, followee_id	Counts for likes, comments, followers, posts	Left joins preserve every user	Engagement and per-post rates; remove extreme outliers	80/20 split; random forest models; classification metrics	Interactive inputs; confusion matrix; scatter plots

Key design choice	The notebook uses left joins beginning with the users table. This keeps users with no observed likes, comments, posts, or followers; missing aggregates are converted to 0.
-------------------	---

2. Dataset and Data Model

The project uses a normalized Instagram-like relational dataset. The recorded dataset contains 100 users, 257 photos, 1,000 likes, 1,000 comments, 1,000 follow relationships, 501 photo-tag links, and 21 tags.

File	Rows	Important fields	How it is used
users.csv	100	id, username, created_at	Base user table for both modelling pipelines.
photos.csv	257	id, user_id, image_url, created_at	Counts posts and maps reactions back to content creators.
likes.csv	1,000	user_id, photo_id, created_at	Counts actions by users; also joins through photo to count likes received.
comments.csv	1,000	id, comment_text, user_id, photo_id, created_at	Counts actions by users; also joins through photo to count comments received.
follows.csv	1,000	follower_id, followee_id, created_at	Counts followers for each followee.
photo_tags.csv	501	photo_id, tag_id	Available in the data but not used in the current models.
tags.csv	21	id, tag_name, created_at	Available for hashtag/tag analysis; not used in the current models.

Relationship map

users.id links to photos.user_id (creator), likes.user_id (liker), comments.user_id (commenter), follows.follower_id, and follows.followee_id. **photos.id** links to likes.photo_id, comments.photo_id, and photo_tags.photo_id. **tags.id** links to photo_tags.tag_id.

Important terminology

The notebook uses “likes/comments by a user” for the classifier, but “likes/comments received on a user’s photos” for the regression pipeline. This difference is intentional in the code and should be explained clearly.

3. Module A - Influencer Classification

This module is an engagement-threshold classifier. It first makes a user-level table from **users**, then counts each user's records in likes and comments. Missing counts become zero.

Feature engineering and label

Item	Definition in the notebook	Why it matters
total_likes	Number of like records where the user is the liker.	Measures a user's liking activity.
total_comments	Number of comment records where the user is the commenter.	Measures a user's commenting activity.
engagement	total_likes + total_comments	Combined activity score.
influencer (target)	1 when engagement > median engagement; otherwise 0.	Creates a balanced-ish binary label from the project dataset.

Baseline before machine learning

The notebook includes a simple rule-based prediction: receive likes and comments as input, add them, and compare the total with the dataset median engagement. Above the median gives "Influencer"; otherwise "Not Influencer." This provides a transparent baseline.

ML pipeline

- Inputs X: **total_likes** and **total_comments**. Target y: **influencer**.
- Training/testing: **train_test_split** with test_size=0.20 and random_state=42.
- Algorithm: **RandomForestClassifier()**. A random forest aggregates decisions from multiple decision trees.
- Output: binary prediction, then the notebook prints accuracy and a classification report.

Recorded evaluation result

The saved notebook output reports accuracy = **1.00** on 20 test users. For class 0, precision/recall/F1 are all 1.00 (support 12); for class 1, all are 1.00 (support 8).

Why this result must be interpreted carefully

The target is calculated directly from the same two input features: engagement = total_likes + total_comments, then influencer is whether engagement is above its median. This makes the classification task close to learning a known threshold from its own ingredients. The 100% score confirms the code pipeline on this split; it does **not** demonstrate reliable real-world influencer prediction. A production label should come from an independent business definition, such as verified status, campaign performance, or a future engagement outcome.

4. Module B - Per-Post Engagement Forecasting

This module changes perspective from user actions to content-creator outcomes. Likes and comments are joined to photos by photo_id, then grouped by the photo owner. This produces engagement received by each creator.

Data preparation

- Count followers by grouping follows on **followee_id**.
- Count posts by grouping photos on **user_id**.
- Join likes and comments to photos; group by the photo owner to count likes and comments received.
- Left-merge all aggregates onto users and fill missing values with zero.
- Create rate targets: **likes_per_post = likes / (posts + 1)** and **comments_per_post = comments / (posts + 1)**. The +1 prevents division by zero.
- Remove very high values using the 99th percentile rule independently for likes_per_post and comments_per_post.

Models

Model	Inputs	Target	Purpose
likes_model: RandomForestRegressor	followers, posts	likes_per_post	Estimate average likes received per post.
comments_model: RandomForestRegressor	followers, posts	comments_per_post	Estimate average comments received per post.

Prediction output

The user enters follower count and total posts. The notebook predicts both rate values, converts them to integers, and applies display floors: **max(50, predicted likes)** and **max(2, predicted comments)**. These floors are presentation rules, not part of the learned random forest model.

Evaluation status

Unlike the classifier, the notebook trains the two regressors but does **not** calculate or print MAE, RMSE, or R-squared. Do not claim regression accuracy. The correct next step is to evaluate both models on yL_test and yC_test using MAE/RMSE/R2.

5. Technology Stack and Evaluation

Tool / technology	Role in the project	How to explain it
Python	Programming language for analysis and machine learning.	It coordinates reading data, transformations, modelling, and outputs.
Pandas	CSV loading, grouping, merging, filling missing values, quantiles.	It turns event-level tables into a clean user-level model table.
NumPy	Imported numerical computing library.	Supports numerical data work; not heavily used in the shown notebook cells.
scikit-learn	train_test_split, random forest classifiers/regressors, metrics.	It provides the reusable machine-learning training and evaluation APIs.
Matplotlib	Plots confusion matrix and scatter charts.	It visualizes model and engagement patterns.
Seaborn	Imported visualization library.	Available for styling/statistical charts; not actively used in the shown charts.
Jupyter Notebook (.ipynb)	Interactive project workspace.	Keeps code, results, prompts, and visual output together.
CSV files	Local relational dataset.	Simple structured source data that is easy to inspect and reproduce.

Metrics and what they mean

Metric	Used?	Meaning
Accuracy	Yes - classifier	Share of all test predictions that were correct.
Precision	Yes - classifier report	Of predicted influencers, how many were actually labelled influencer?
Recall	Yes - classifier report	Of actual labelled influencers, how many were detected?
F1-score	Yes - classifier report	Balances precision and recall.
Confusion matrix	Yes - classifier plot	Shows true negatives, false positives, false negatives, and true positives.
MAE / RMSE / R2	Not yet implemented for regressors	Recommended to quantify per-post forecast error and explained variance.

6. Analytics, Outputs, and Module Checklist

The notebook has three visualization outputs:

Visualization	Axes / contents	Interpretation
Confusion Matrix	Actual labels vs predicted classifier labels; cells labelled TN, FP, FN, TP.	Checks types of correct and incorrect classification decisions.
Followers vs Likes scatter plot	x: followers; y: total likes received.	Explores whether accounts with more followers also receive more likes.
Posts vs Engagement scatter plot	x: number of posts; y: total likes received + total comments received.	Explores content volume versus received engagement.

Specific modules implemented

- **File discovery:** lists CSV files under the inputs folder.
- **Data ingestion:** reads users, photos, likes, comments, and follows into Pandas DataFrames.
- **User activity aggregation:** computes likes/comments made by each user.
- **Rule-based classifier:** compares supplied engagement with the median threshold.
- **Random forest classifier:** trains a binary influencer-status model.
- **Creator outcome aggregation:** counts followers, posts, likes received, and comments received.
- **Engagement-rate engineering:** calculates likes/comments per post and filters extreme values.
- **Two regression models:** forecasts likes per post and comments per post.
- **Interactive prediction:** accepts values through input prompts.
- **Visual analytics:** produces confusion matrix and exploratory scatter plots.

What is not implemented yet

- No persistent model saving/loading (for example, joblib or pickle).
- No web, mobile, or dashboard user interface; interaction occurs inside the notebook.
- No model hyperparameter tuning, cross-validation, feature importance analysis, or regression metrics.
- Tags/photo_tags are present but unused; they are a strong opportunity for content-topic features.

7. Limitations, Improvements, and Honest Project Positioning

A strong project explanation includes both what works and what you would improve. This project is a well-structured learning pipeline for social-media data aggregation and random forests, but it is not yet a production influencer scoring system.

Current limitation	Why it matters	Practical improvement
Derived target leakage in classification	The target is a threshold of the same likes/comments supplied as features; high performance is expected.	Use an independent label or predict future-period engagement from past-period features.
Small dataset	100 users makes a single 80/20 split unstable and limits generalization.	Collect more accounts; use stratified cross-validation and a holdout set.
No regression metrics	Forecast quality cannot be quantified.	Report MAE, RMSE, R2, and a baseline comparison for each target.
Limited inputs	Only followers and posts feed the regressors.	Add account age, posting frequency, recency, tags, content category, time, and prior engagement.
Outlier removal rule	The 99th-percentile filter may remove useful high-performing accounts.	Document retained rows, compare robust alternatives, and fit transformations.
Hard-coded output floors	min 50 likes / 2 comments can hide a low model forecast.	Show raw prediction and label any business rule separately.
Reproducibility	Random forest objects have no random_state in the notebook.	Set random_state=42 and record package versions.

Recommended next iteration

Build a time-aware dataset: use engagement from a past window to predict engagement in a future window. Add tags and photo-level features, use cross-validation, compare random forest against a simple baseline, calculate regression metrics, and present results in a lightweight Streamlit dashboard.

8. How to Explain the Project

30-second version

“I built an Instagram analytics and machine-learning project using Python. I combined user, post, like, comment, and follow data with Pandas to create engagement features. I used a Random Forest classifier to classify users above a median engagement threshold, and two Random Forest regressors to estimate likes and comments per post from followers and posting history. I also added a confusion matrix and scatter plots for analysis. The project taught me data joining, feature engineering, model training, and responsible evaluation.”

60-90 second self-introduction version

“One project I worked on is an Instagram Analytics and Influencer Prediction system. The goal was to understand social-media activity from connected CSV tables and convert it into useful predictions. I started with users as the base table and joined aggregated likes, comments, photos, and follow relationships using Pandas. I created engagement measures such as total likes, total comments, followers, posts, likes per post, and comments per post. For classification, I used a RandomForestClassifier to label users as influencer or not influencer based on an engagement threshold. For forecasting, I trained separate RandomForestRegressor models to estimate likes per post and comments per post using follower and post counts. I evaluated the classification model with accuracy, precision, recall, F1-score, and a confusion matrix, and created scatter plots to inspect relationships. One important lesson was that the classification label came from the same engagement features, so a perfect score should be treated as a pipeline demonstration, not a real-world performance claim. My next step would be independent labels, time-based features, and regression metrics.”

Likely questions and concise answers

Question	Answer
Why Random Forest?	It handles nonlinear relationships, needs little feature scaling, and can capture interactions between features. It was a sensible baseline for a small tabular dataset.
What is feature engineering here?	Converting raw events into user-level signals: counts of likes, comments, followers, posts, and per-post engagement rates.
Why left join and fillna(0)?	To retain every user, including inactive users; no recorded event means zero observed count in this dataset.
What does the classifier actually predict?	Whether the user's activity-based engagement is above the dataset median, not verified commercial influence.
What caused 100% accuracy?	The label was derived from the same two input values, so the model effectively learned the rule used to make the label.
How would you evaluate regressors?	Use MAE for average absolute error, RMSE to penalize large errors, R2 for variance explained, plus a simple baseline.
What would you add next?	Time-based splits, independent future outcome labels, tag/content features, cross-validation, model persistence, and a dashboard.

9. Prompt to Use This PDF with Another AI

Attach this PDF and paste the following instruction:

Copy-ready instruction

Use this document as the source of truth for my Instagram Analytics project. First, quiz me with one question at a time, starting easy and becoming more technical. After each answer, assess it, correct inaccuracies, and give me a better interview-ready answer. Teach the technologies used (Python, Pandas, scikit-learn, random forest, evaluation metrics, Matplotlib) in the context of this project. Help me prepare a 30-second and 90-second project explanation for self-introduction. Be especially strict about the difference between the engagement-based classifier and real-world influencer prediction, and do not claim regression metrics that are not documented.

Facts an AI must preserve

- This is a Jupyter/Python project using local CSV data and Pandas aggregations.
- The classifier uses total_likes and total_comments; the target is above-median engagement derived from their sum.
- Recorded classifier evaluation is 1.00 accuracy on 20 test users; interpret it as pipeline validation with target leakage risk.
- The regression models use followers and posts to estimate likes_per_post and comments_per_post.
- Regression metrics are not present in the notebook; they must be proposed, not invented.
- The project includes confusion-matrix and scatter-plot visualizations.
- Tags and photo_tags exist in the dataset but are not used in the shown model code.

Final takeaway

Present this as an Instagram analytics learning project with clear feature engineering and machine-learning workflows. Being precise about limitations will make your explanation stronger and more credible.